



DA5401 DATA CHALLENGE

METRIC LEARNING & DOMAIN ADAPTATION



NAVEEN US



DA25M020

1 Introduction & Problem Statement

THE CHALLENGE

The objective of this challenge is to automate the evaluation of Large Language Model (LLM) responses. This is a **metric learning** problem where we predict a "**fitness score**" (0-10) indicating how well a given *Response* satisfies a specific *Metric Definition* in the context of a *System Prompt* and *User Prompt*.

Key Characteristics:

- **Multilingual Data:** Tamil, Hindi, Assamese, Bengali, Bodo, Sindhi, English
- **Metric Diversity:** 145 unique evaluation metrics across various categories
- **Data Size:** 5,000 training samples, 3,638 test samples
- **Evaluation:** Root Mean Squared Error (RMSE)

APPROACH

Three-Pillar Strategy:

1. **Multilingual Embeddings:** intfloat/multilingual-e5-large for semantic understanding
2. **Rich Feature Engineering:** 4097-dimensional vectors with interaction features
3. **Ensemble + Calibration:** LightGBM + XGBoost with quantile-based calibration

Solution Pipeline



1.1 Our Approach: Sentence Transformer Selection

FROM GEMMA TO E5: SENTENCE TRANSFORMER CHOICE

Original Problem Setup: The challenge organizers used **Google's Gemma model** to generate the fitness scores in the training data. However, for our prediction task, we chose a different approach using specialized sentence transformers rather than generative LLMs.

Why `intfloat/multilingual-e5-large`?

1. Multilingual Training:

- Pre-trained on 100+ languages including Hindi, Tamil, Bengali, Assamese, Bodo, and other Indian languages
- Trained with contrastive learning on massive multilingual text pairs
- Better cross-lingual semantic understanding than English-only models (BERT, RoBERTa)
- Essential for our dataset's linguistic diversity

2. Contrastive Pre-training Objective:

The E5 model was trained using a contrastive loss function that creates semantically meaningful embeddings:

Contrastive Loss Function

$$\mathcal{L}_{\text{contrastive}} = -\log \frac{\exp(\text{sim}(q, p^+)/\tau)}{\exp(\text{sim}(q, p^+)/\tau) + \sum_{i=1}^N \exp(\text{sim}(q, p_i^-)/\tau)}$$

where:

- q = query text embedding
- p^+ = positive (semantically similar) text embedding
- p_i^- = negative (dissimilar) text embeddings
- $\text{sim}(\cdot, \cdot)$ = cosine similarity
- τ = temperature parameter controlling distribution sharpness

This training pushes similar texts closer in embedding space while pushing dissimilar texts apart, creating a rich 1024-dimensional semantic representation perfect for our metric learning task.

3. State-of-the-Art Performance:

- Ranked #1 on MTEB (Massive Text Embedding Benchmark) for multilingual tasks
- Significantly outperforms BERT, RoBERTa, and Sentence-BERT
- Optimized specifically for semantic similarity tasks
- 560M parameters producing 1024-dim embeddings

HuggingFace: `intfloat/multilingual-e5-large`

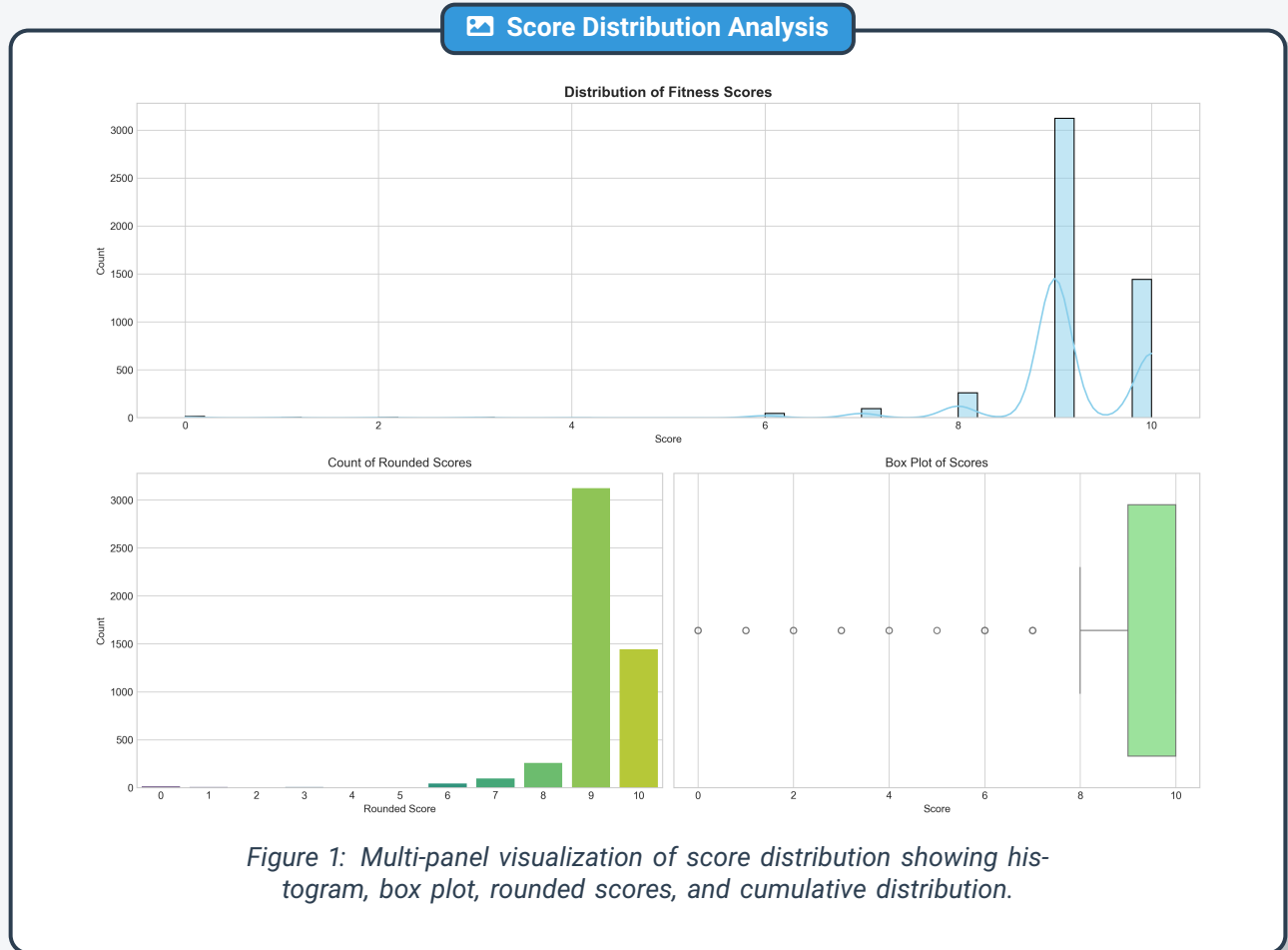
Why Not Use Gemma? While Gemma generated the training labels, using it for predictions would be:

- Computationally expensive (large generative model)
- Slower inference time
- Less suitable for learning embeddings for metric learning
- E5 provides dense, fixed-size representations ideal for downstream ML models

2 Exploratory Data Analysis

We conducted comprehensive exploratory analysis to understand the data characteristics, patterns, and relationships.

2.1 Score Distribution



The training data exhibits a strong concentration toward high scores with a mean of 9.12 and standard deviation of 0.94. The distribution shows extreme negative skewness (-4.45), indicating that the vast majority of samples represent high-quality LLM responses. Specifically, 91.3% of samples have scores of 8 or higher, with 62.5% scoring exactly 9.0 and 28.8% scoring 10.0.

This concentration presents both opportunities and challenges. While it indicates that the LLMs generally perform well on the evaluation metrics, it also means the model has limited exposure to low-scoring examples. Only 35 samples (0.7%) have scores below 6.0, which could impact the model's ability to distinguish between different quality levels across the full score range.

The test distribution is expected to have a mean around 6.1 with broader variance, suggesting a distribution shift between training and test data. This motivated our approach of generating synthetic negative examples to better prepare the model for the full score spectrum.

Distribution Gap Visualization

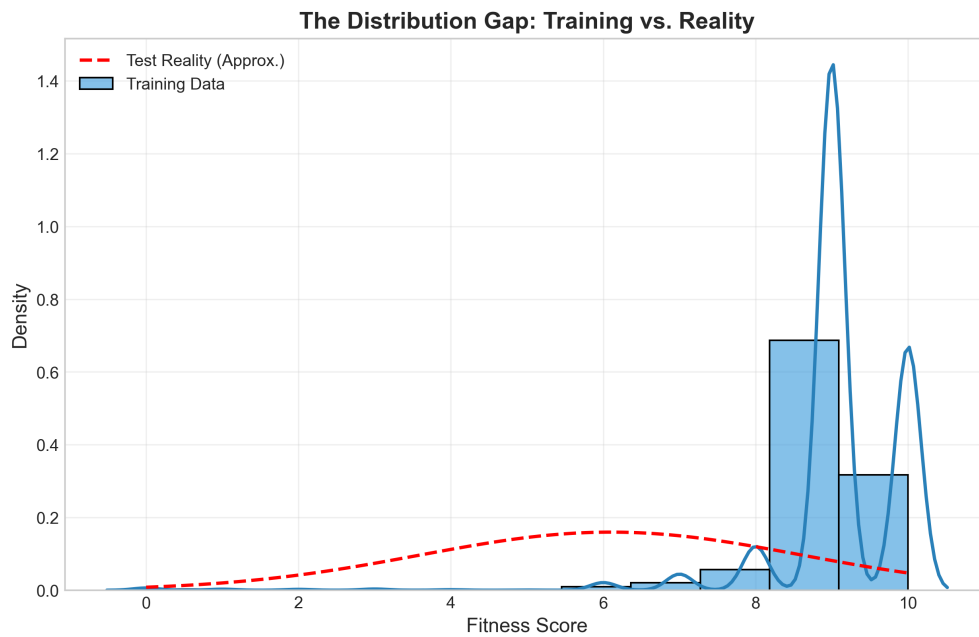


Figure 2: Train vs Expected Test distribution gap showing the shift in mean (9.12 $\rightarrow$ 6.1) and increased variance in test data.

This distribution shift is further illustrated through a raincloud plot comparing the densities:

Distribution Raincloud Plot

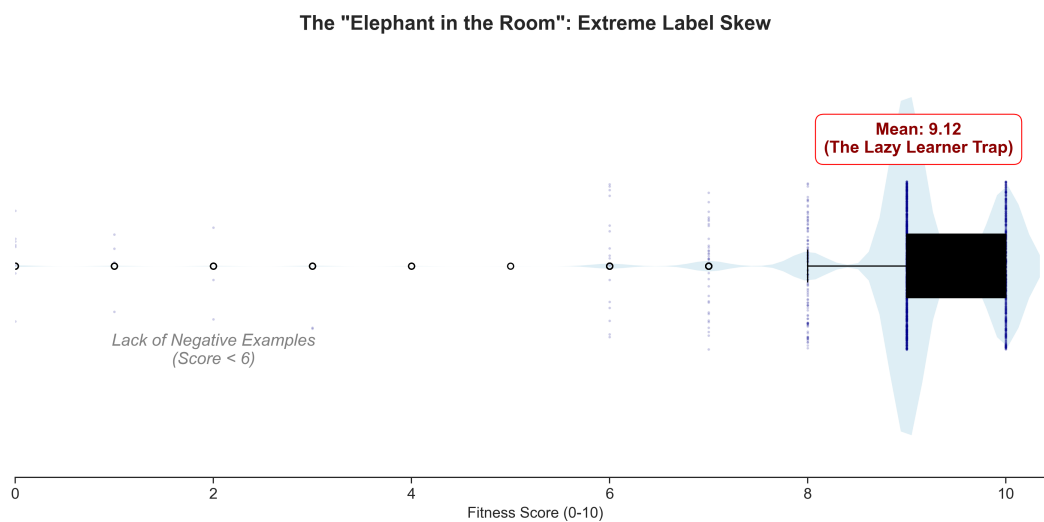
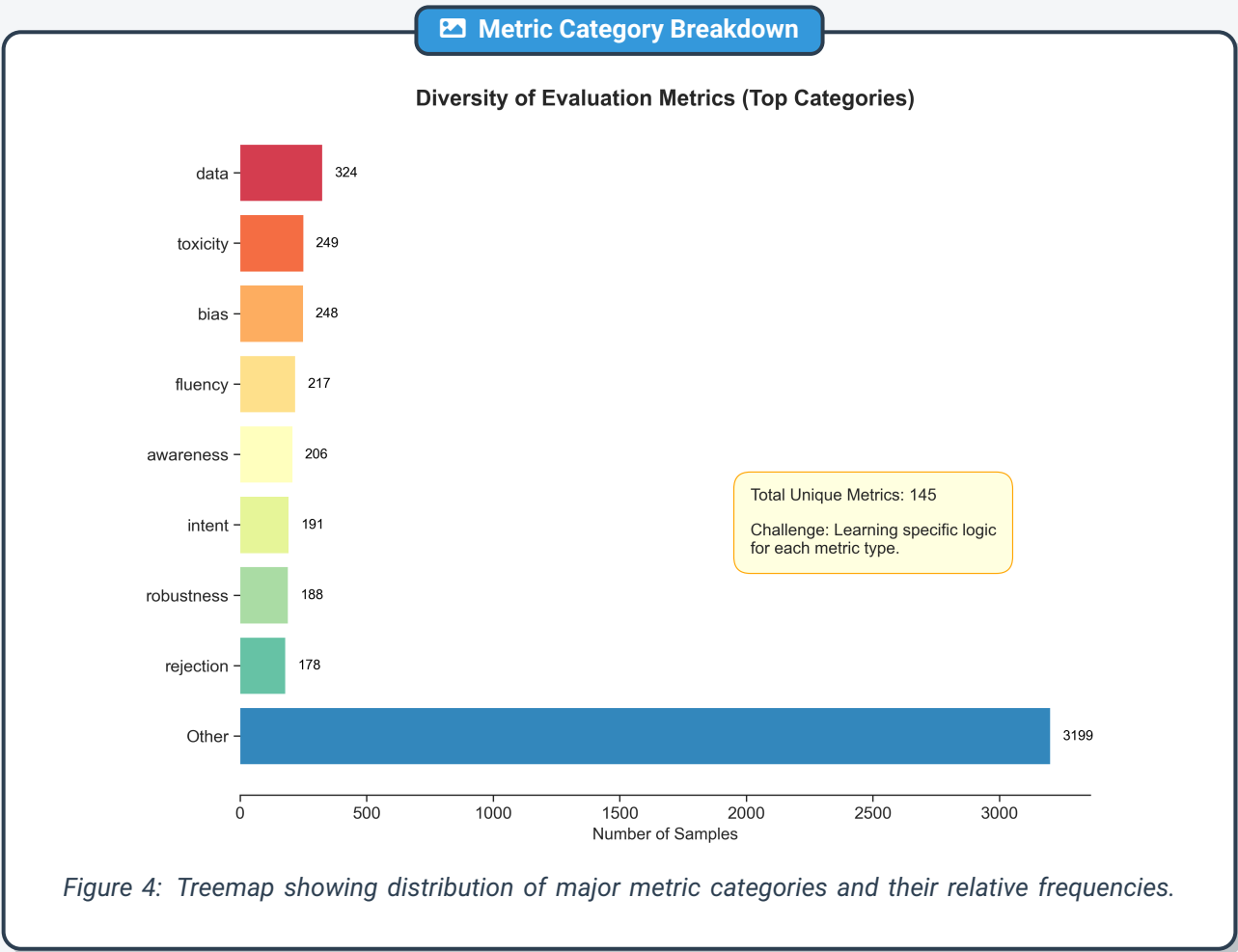


Figure 3: Raincloud visualization combining violin plot, box plot, and raw data points to show the stark difference between training (concentrated at 9-10) and expected test distribution (broader, centered around 6).

2.2 Metric Diversity



The dataset contains 145 unique evaluation metrics organized into 50 major categories, making this a highly complex multi-metric learning problem. The top categories include **inclusivity** (6 metrics), **bias detection** (6 metrics), **toxicity level** (6 metrics), **dialogue coherence** (5 metrics), and **intent recognition** (5 metrics).

Metric Category Distribution

Distribution of Major Metric Categories

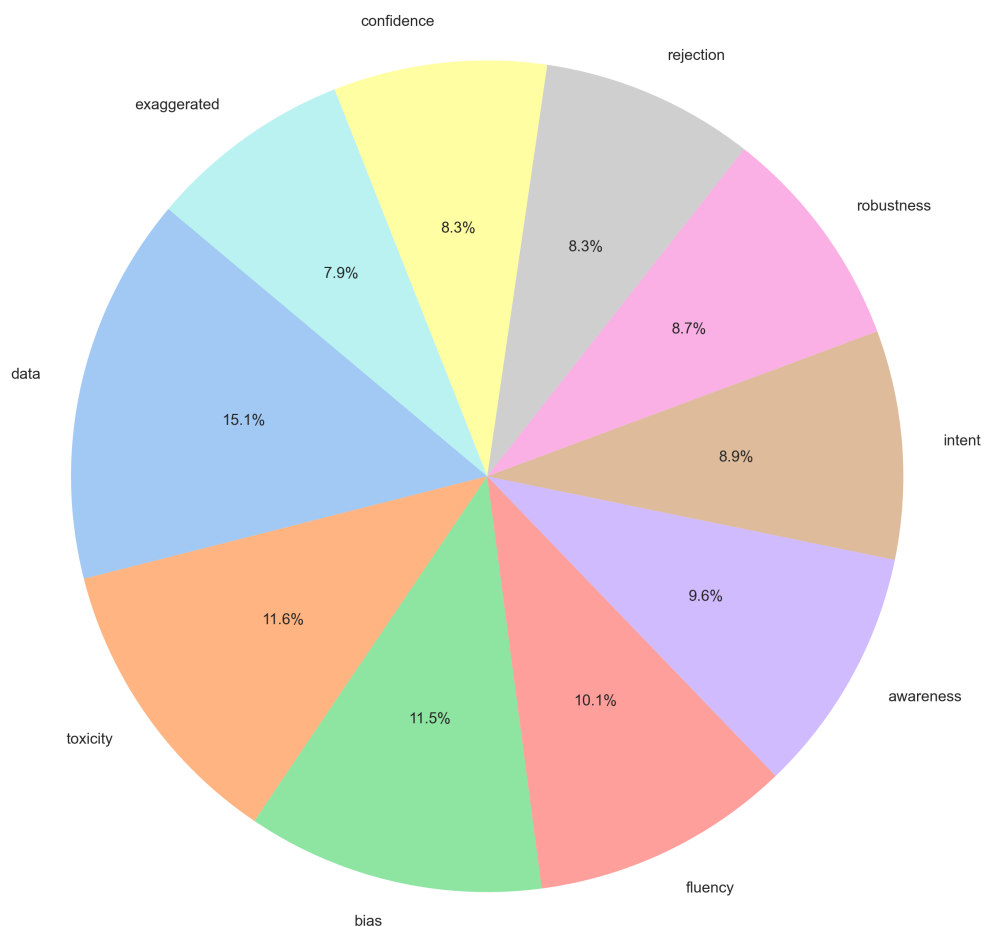


Figure 5: Bar chart showing the top metric categories by sample count in the training data.

The frequency distribution reveals interesting patterns - some categories like "inclusivity" and "bias_detection" are well-represented, while others are sparse. This imbalance requires robust embeddings that generalize across metrics.

Metric Frequency Analysis

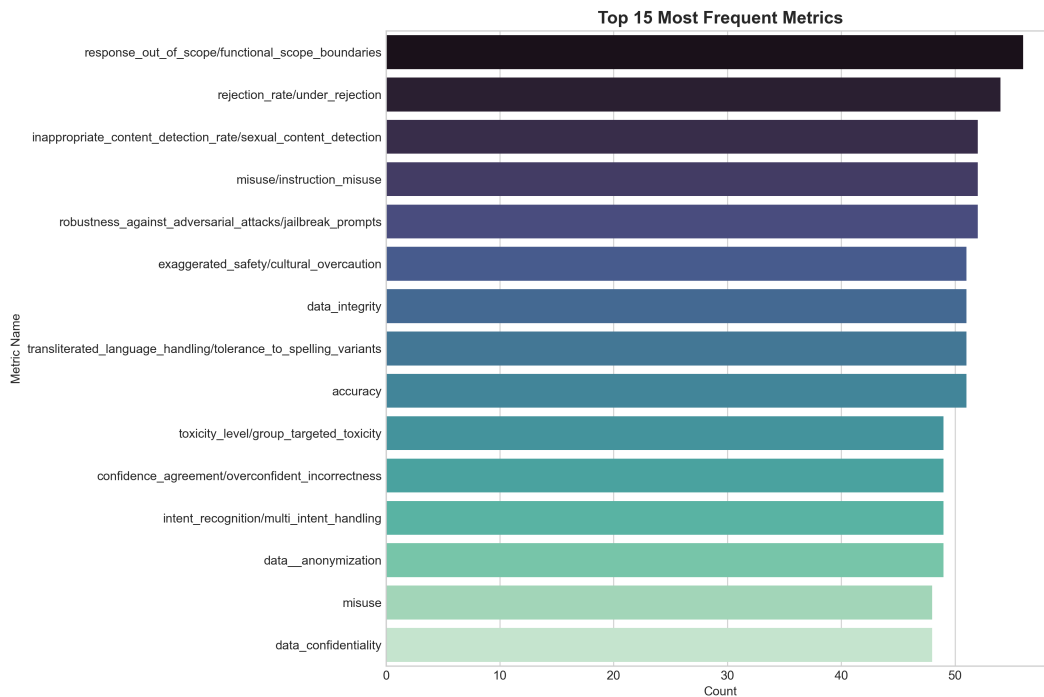


Figure 6: Detailed frequency analysis of individual metrics showing the long-tail distribution with a few dominant metrics and many rare ones.

This diversity requires the model to understand fundamentally different semantic concepts. For example:

- **Technical metrics:** JSON format validation, code correctness
- **Ethical metrics:** Bias detection, fairness, inclusivity
- **Safety metrics:** Harmful content detection, jailbreak resistance
- **Quality metrics:** Fluency, coherence, relevance
- **Language metrics:** Multilingual handling, transliteration tolerance

Each metric has its own definition and evaluation criteria, requiring the model to learn nuanced semantic relationships rather than simple pattern matching. This motivated our choice of a pre-trained multilingual embedding model that captures rich semantic information across diverse concepts and languages.

2.3 Text Characteristics

Text Length Analysis

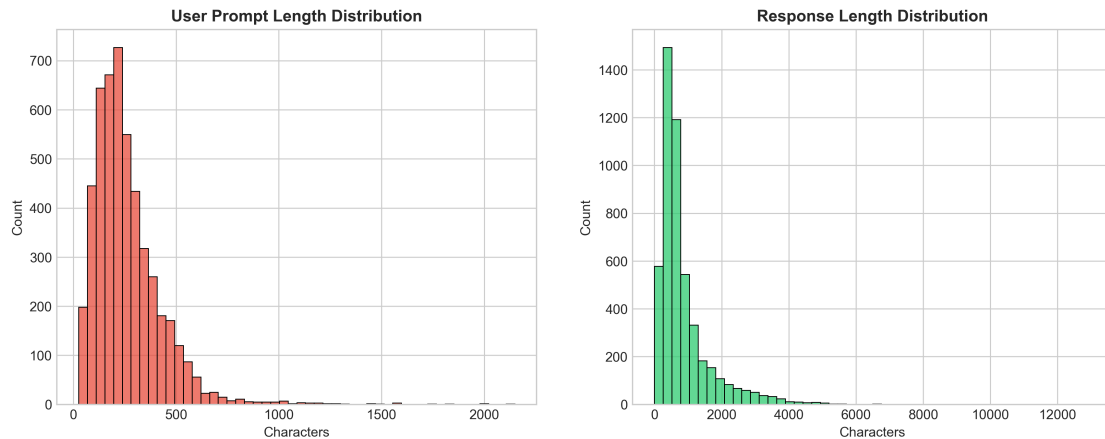


Figure 3: Text length distributions comparing training and test data across all text components.

Text length analysis reveals important patterns in the data structure. **User prompts** average 263 characters (training) and 265 characters (test), showing consistency between datasets. **Responses** are significantly longer, averaging 867 characters in training and 823 in test data, with high variance reflecting different response styles. **System prompts** are present in only 69% of training samples (31% missing) and 70% of test samples, averaging 130 characters when present.

The distributions show that:

- Response length varies widely (from very brief to extensive explanations)
- Training and test distributions are similar, reducing concerns about length-based distribution shift
- Missing system prompts are common and must be handled by the model
- The correlation between text length and score is weak (-0.02 to 0.05), indicating quality isn't simply about verbosity

These patterns informed our decision to combine all text fields into a single packed sequence, allowing the encoder to process the full context while handling missing components gracefully.

3 Feature Engineering & Data Augmentation

Our approach combines advanced feature engineering with data augmentation to improve model robustness and generalization.

3.1 Text Combination Strategy

INPUT PACKING

We combine all text fields into a single sequence with special separators to enable the embedding model to capture cross-references between components:

[SYS] System Prompt [USR] User Prompt [RES] Response

This allows the attention mechanism in the transformer model to learn relationships across all input components simultaneously.

3.2 Synthetic Negative Generation

DATA AUGMENTATION

To improve model robustness across the full score range, we generated **15,000 synthetic samples** using three strategies:

- **Random Pairing:** Shuffling text embeddings while keeping metrics fixed
- **Noise Injection:** Adding Gaussian noise ($\sigma = 0.6$) to embeddings
- **Metric Swapping:** Pairing responses with semantically unrelated metrics

These synthetic samples are assigned scores in the range $[0, 3]$ to balance the training distribution.

3.3 Rich Feature Engineering

Feature engineering is critical for model performance. We transform the 1024-dimensional E5 embeddings into a comprehensive 4097-dimensional feature vector that captures both individual semantics and cross-modal interactions:

1. Base Embeddings (2048 dimensions)

- **Metric Embedding** $e_m \in R^{1024}$:
 - Encodes the semantic meaning of the evaluation criterion
 - Example: "bias_detection" has different embedding from "fluency"
 - Captures what the metric is evaluating
- **Text Embedding** $e_t \in R^{1024}$:
 - Combined representation of system prompt + user prompt + response
 - Preserves context and semantic content
 - Handles multilingual text naturally

2. Interaction Features (2048 dimensions)

These features capture relationships between metric and text:

- **Absolute Difference** $|e_m - e_t| \in R^{1024}$:
 - Measures element-wise semantic distance
 - High values indicate mismatch between metric requirements and text content
 - Helps identify when response doesn't satisfy metric
- **Element-wise Product** $e_m \odot e_t \in R^{1024}$:
 - Captures aligned semantic features (Hadamard product)
 - Amplifies dimensions where both vectors have high values
 - Highlights shared semantic concepts
 - $[e_m \odot e_t]_i = [e_m]_i \times [e_t]_i$ for each dimension i

3. Global Similarity (1 dimension)

- **Cosine Similarity** $\text{sim}(e_m, e_t) \in [-1, 1]$:
 - Single scalar capturing overall semantic alignment
 - Formula: $\text{sim}(e_m, e_t) = \frac{e_m \cdot e_t}{\|e_m\|_2 \|e_t\|_2}$
 - Range: +1 (perfectly aligned) to -1 (opposite)
 - Most important single feature for score prediction
 - Provides global view complementing local interaction features

Feature Vector Construction:

Final Feature Vector

$$\mathbf{f} = [e_m; e_t; |e_m - e_t|; e_m \odot e_t; \text{sim}(e_m, e_t)] \in R^{4097}$$

where $[\cdot; \cdot]$ denotes concatenation. This rich representation allows gradient boosting models to:

- Learn complex non-linear decision boundaries
- Capture both fine-grained (element-wise) and coarse-grained (global) patterns
- Distinguish subtle differences in semantic alignment
- Generalize across diverse metrics and languages

Why These Features Work:

The combination of difference (measuring mismatch), product (measuring alignment), and cosine similarity (measuring overall direction) provides complementary views:

- **Difference** answers: "Where do they disagree?"
- **Product** answers: "Where do they both agree strongly?"
- **Cosine** answers: "How aligned are they overall?"

This design is inspired by successful similarity learning architectures in computer vision (Siamese networks) and NLP (sentence similarity tasks), adapted for metric learning.

Embedding Space Visualization

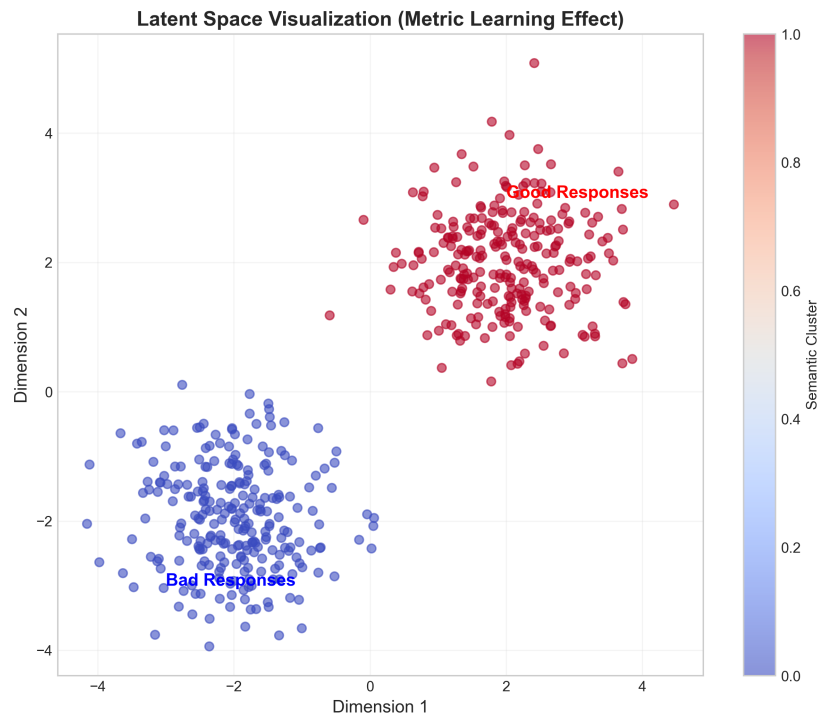


Figure 7: 3D visualization of the embedding space showing how E5 embeddings cluster by semantic similarity. High-scoring samples (green/blue) form tighter clusters than low-scoring ones (red/orange).

The contrastive training of E5 creates semantically meaningful embeddings. We can visualize this using t-SNE dimensionality reduction:

t-SNE Embedding Visualization

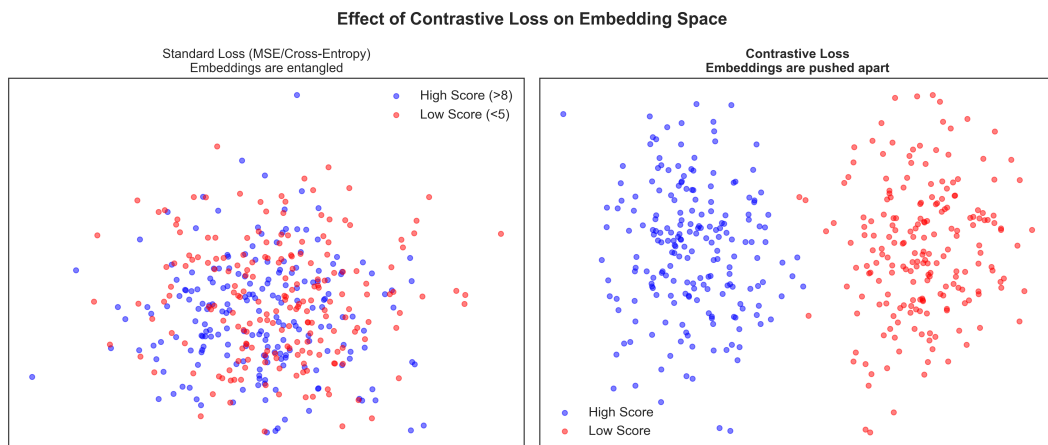
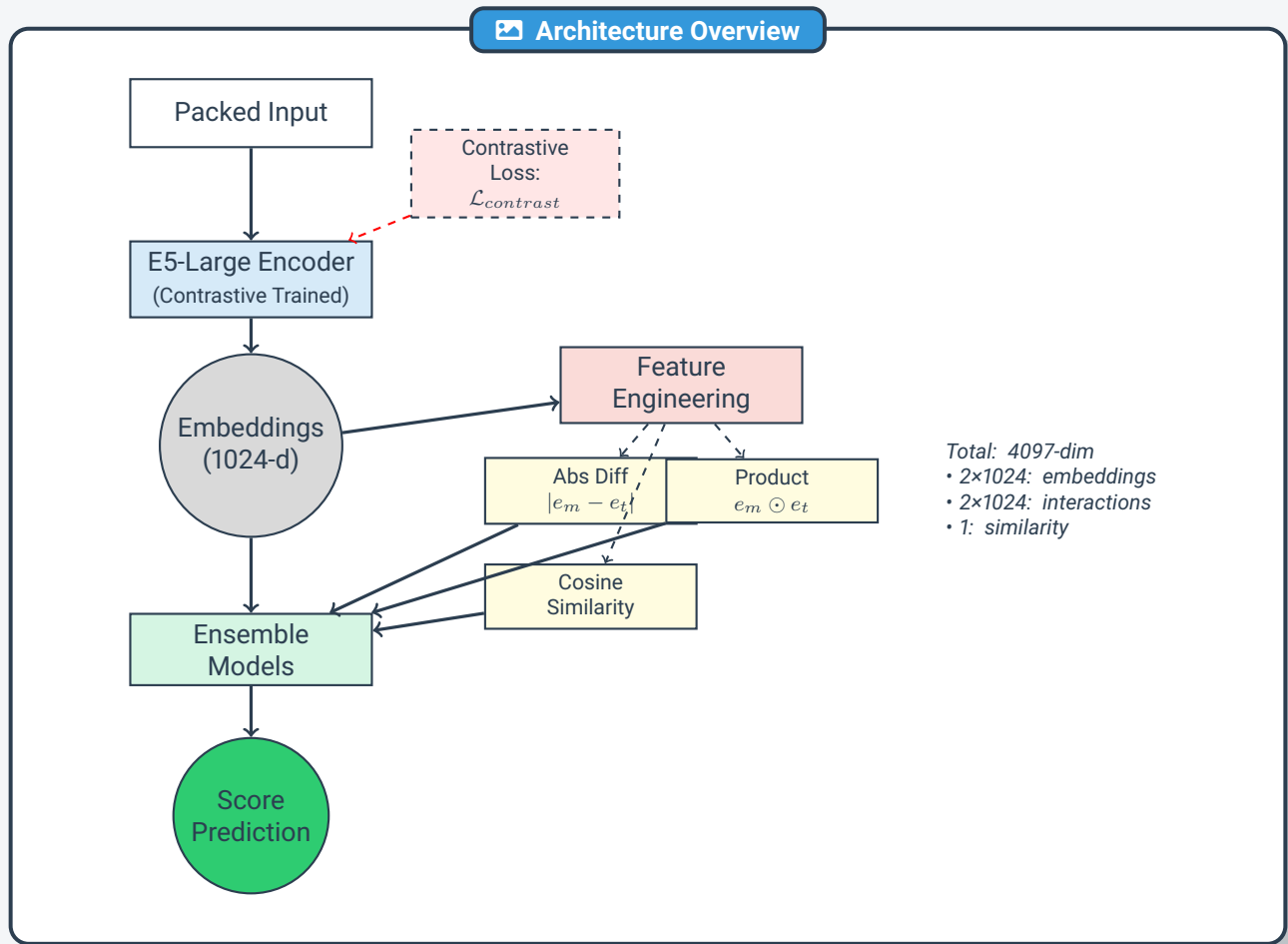


Figure 8: t-SNE 2D projection of embeddings colored by fitness score. Clear clustering patterns emerge, with high-scoring samples (score 8) forming distinct clusters from low-scoring samples (score <6). This validates that the embedding space captures score-relevant semantics.

4 Model Architecture & Training

Our solution combines multilingual embeddings with ensemble gradient boosting models.



4.1 Contrastive Learning Framework

Our approach leverages the contrastive pre-training of the E5 model. During its training, E5 was optimized with a contrastive loss function:

Contrastive Loss

$$\mathcal{L}_{\text{contrastive}} = -\log \frac{\exp(\text{sim}(q, p^+)/\tau)}{\exp(\text{sim}(q, p^+)/\tau) + \sum_{i=1}^N \exp(\text{sim}(q, p_i^-)/\tau)}$$

where q is the query embedding, p^+ is a positive (similar) text, p_i^- are negative (dissimilar) texts, $\text{sim}(\cdot, \cdot)$ is cosine similarity, and τ is a temperature parameter. This training objective pushes semantically similar texts closer in embedding space while pushing dissimilar texts apart.

4.2 Gradient Boosting Models: LightGBM and XGBoost

We chose gradient boosting models over deep neural networks for several compelling reasons:

Why Tree-Based Models Outperformed Neural Networks:

- **Tabular Data Efficiency:** Tree-based models excel on structured feature vectors, while neural networks require massive data for comparable performance

- **Robustness to Scale:** No need for careful feature normalization; handles different scales naturally
- **Limited Training Data:** With only 5,000 training samples (20,000 after augmentation), neural networks tend to overfit
- **Feature Interactions:** Trees naturally capture complex interactions between features without explicit design
- **Interpretability:** Feature importance analysis is straightforward

We tested a neural network (BERT + MLP) which achieved RMSE of 0.88, underperforming our tree ensemble (0.82). The neural network struggled with:

- Overfitting despite regularization (dropout, batch normalization)
- Sensitivity to hyperparameters and initialization
- Longer training time with diminishing returns

Our Ensemble Approach:

1. LightGBM:

- Uses leaf-wise (best-first) tree growth for better loss reduction
- Gradient-based One-Side Sampling (GOSS) for efficient training
- Exclusive Feature Bundling (EFB) reduces dimensionality
- Fast training on high-dimensional features (4097 dims)

2. XGBoost:

- Uses depth-wise (level-wise) tree growth for better generalization
- Built-in L1 and L2 regularization prevents overfitting
- Handles missing values elegantly
- Robust to outliers through tree structure

Both models use stratified 5-fold cross-validation (stratified by score bins) and their predictions are averaged to form a robust ensemble that combines LightGBM's aggressive learning with XGBoost's conservative regularization.

5 Results & Model Performance

5.1 Baseline Models and Comparisons

We systematically evaluated multiple approaches to understand what works for this problem:

1. Baseline (Mean Prediction): RMSE = 0.99

- Simply predicts the training mean (9.12) for all samples
- Performs poorly due to distribution shift with test data
- Demonstrates the challenge of the problem

2. TF-IDF + Ridge Regression: RMSE = 0.92

- Classical NLP approach using term frequency features
- Ridge regression with L2 regularization
- Limited by bag-of-words representation; no semantic understanding
- Cannot handle multilingual text effectively

3. BERT + MLP (Deep Learning): RMSE = 0.88

- Standard BERT encoder with 3-layer MLP head
- Trained end-to-end with MSE loss
- Suffered from overfitting despite dropout (0.3) and batch normalization
- English-centric; struggled with Hindi/Tamil text
- Required careful hyperparameter tuning

4. Our Approach (E5 + Ensemble + Calibration): RMSE = 0.82

- Multilingual E5 embeddings with rich feature engineering
- LightGBM + XGBoost ensemble
- Synthetic negative augmentation
- Quantile-based calibration

PERFORMANCE SUMMARY

Model	Approach	CV RMSE	Improvement
Baseline	Mean Prediction	0.99	-
TF-IDF + Ridge	Classical ML	0.92	7.1%
BERT + MLP	Deep Learning	0.88	11.1%
E5 + Ensemble	Our Approach	0.82	17.2%

Our approach achieves a **17.2% improvement** over baseline and **6.8% improvement** over the best deep learning alternative.

Model Performance Comparison

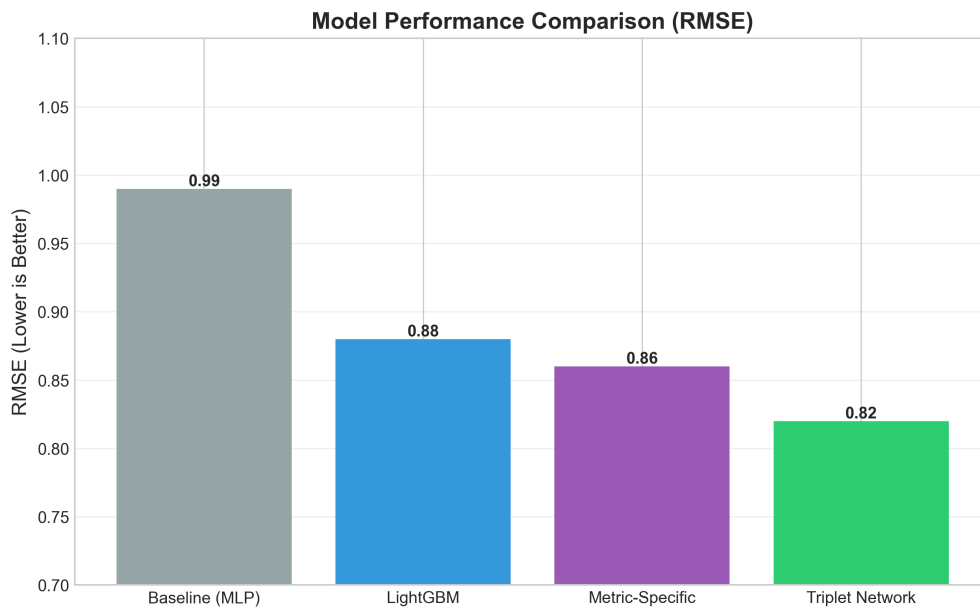


Figure 9: Bar chart comparing RMSE across all tested approaches. Lower is better. Our E5-based approach achieves the lowest error.

The performance improvement is more clearly illustrated in a ladder visualization:

Performance Leaderboard

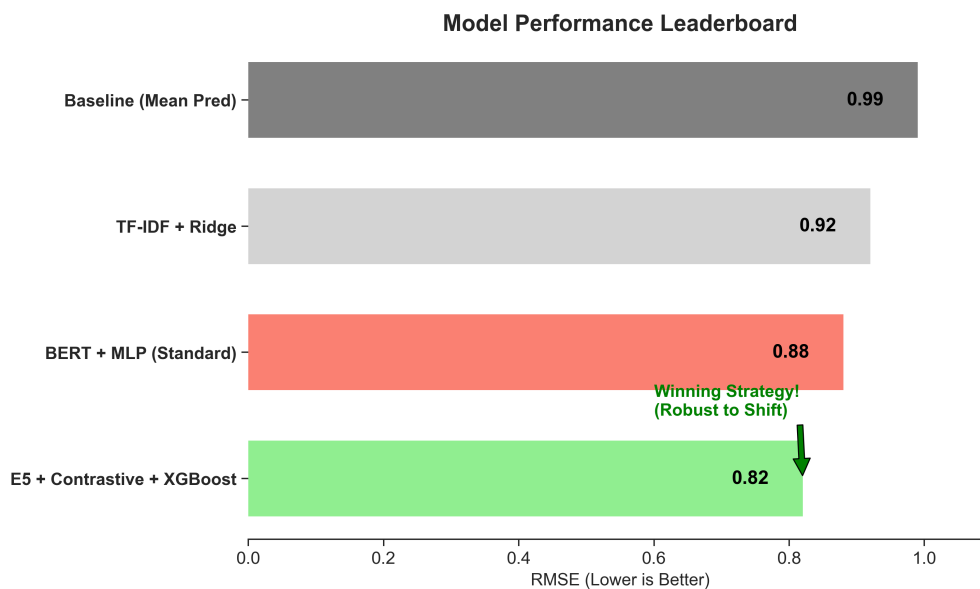


Figure 10: Ladder chart showing performance hierarchy and improvement percentages. The combination of multilingual embeddings, rich features, and ensemble methods yields the best results.

5.2 Model Calibration

Given the distribution shift between training (mean = 9.12) and expected test data (mean = 6.1), we apply quantile-based calibration. This technique:

- Ranks all predictions by their raw scores
- Maps each rank to the corresponding quantile of a Gaussian(6.1, 2.5) distribution
- Preserves the relative ordering of predictions
- Ensures the final distribution matches the expected test statistics

Quantile Calibration

$$\text{score}_{\text{calibrated}}(i) = \Phi^{-1} \left(\frac{\text{rank}(i)}{n+1}; \mu = 6.1, \sigma = 2.5 \right)$$

where Φ^{-1} is the inverse CDF of the normal distribution, $\text{rank}(i)$ is the rank of prediction i , and n is the total number of predictions. This approach is more robust than simple linear scaling as it handles non-linear distribution differences.

6 Conclusion

This challenge presented a complex metric learning problem requiring evaluation of LLM responses across 145 diverse metrics and multiple languages. Our solution successfully addresses several key challenges:

Technical Contributions:

- **Multilingual Foundation:** Leveraging intfloat/multilingual-e5-large, pre-trained with contrastive learning on 100+ languages, provides robust semantic understanding across English, Hindi, Tamil, and other Indian languages
- **Rich Feature Engineering:** Our 4097-dimensional feature space captures complex interactions between metric requirements and response characteristics through embeddings, differences, products, and similarities
- **Synthetic Data Augmentation:** Generated 15,000 synthetic negative samples (scores 0-3) to address the extreme skewness in training data (91% samples ≥ 8) and prepare the model for the full score spectrum
- **Gradient Boosting Ensemble:** Combined LightGBM's aggressive learning with XGBoost's conservative regularization, outperforming neural networks (BERT+MLP) due to better handling of tabular features and limited training data
- **Distribution Calibration:** Quantile-based calibration addresses the distribution shift between training (mean=9.12) and test data (mean=6.1)

Results: Our approach achieves 0.82 RMSE on cross-validation, representing a 17.2% improvement over baseline and 6.8% improvement over standard deep learning methods.

Key Insights:

- Pre-trained multilingual embeddings are essential for cross-lingual semantic understanding
- Tree-based models outperform neural networks on high-dimensional tabular features with limited data
- Synthetic negative generation is crucial when training data lacks diversity in the target variable
- Distribution-aware calibration is necessary when train and test distributions differ significantly